

EXP 09

First Fit & Best Fit Memory Allocation

AIM:

To write a C program for implementing memory allocation methods for fixed partitions using first fit and implement Best Fit memory allocation technique using Python.

1. First Fit Memory Allocation

ALGORITHM

1. Start
2. Get number of memory blocks
3. Get size of each memory block
4. Get number of processes
5. Get size of each process
6. Check each block sequentially
7. Allocate process to the first suitable block
8. Mark block as allocated
9. Display allocation result
10. End

CODE:

```
#include <stdio.h>
```

```
int main() {  
    int blockSize[20], processSize[20];  
    int allocation[20];  
    int m, n, i, j;  
  
    printf("Enter number of memory blocks: ");  
    scanf("%d", &m);  
  
    printf("Enter size of memory blocks:\n");  
    for(i = 0; i < m; i++) {
```

```
    scanf("%d", &blockSize[i]);  
}
```

```
printf("Enter number of processes: ");  
scanf("%d", &n);
```

```
printf("Enter size of processes:\n");  
for(i = 0; i < n; i++) {  
    scanf("%d", &processSize[i]);  
    allocation[i] = -1;  
}
```

```
for(i = 0; i < n; i++) {  
    for(j = 0; j < m; j++) {  
        if(blockSize[j] >= processSize[i]) {  
            allocation[i] = j;  
  
            blockSize[j] -= processSize[i];  
            break;  
        }  
    }  
}
```

```
printf("\nProcess No\tProcess Size\tBlock No\n");
```

```
for(i = 0; i < n; i++) {  
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);
```

```
    if(allocation[i] != -1)  
        printf("%d\n", allocation[i] + 1);  
    else  
        printf("Not Allocated\n");  
}
```

```
return 0;  
}
```

2. Best Fit Memory Allocation

ALGORITHM

1. Start
2. Get number of memory blocks
3. Get size of each memory block
4. Get number of processes
5. Get size of each process
6. Search all blocks for smallest suitable block
7. Allocate process to best fitting block
8. Update remaining memory size
9. Display allocation result
10. End

CODE:

```
#include <stdio.h>
```

```
int main() {
    int blockSize[20], processSize[20];
    int allocation[20];
    int m, n, i, j, bestIdx;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    printf("Enter size of memory blocks:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter size of processes:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
        allocation[i] = -1;
    }
}
```

```

}

for(i = 0; i < n; i++) {
    bestIdx = -1;

    for(j = 0; j < m; j++) {
        if(blockSize[j] >= processSize[i]) {

            if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                bestIdx = j;
            }
        }
    }

    if(bestIdx != -1) {
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}

printf("\nProcess No\tProcess Size\tBlock No\n");

for(i = 0; i < n; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

return 0;
}

```

RESULT:

Thus, First Fit and Best Fit memory allocation techniques were successfully implemented and executed.